

Autonomous Unitree Go2 Control with ArUco Markers and OpenCV

Course Module Plan - Revised v2

Updated with the confirmed student setup flow, current script names, markers 0-4, and confirmed troubleshooting notes.

Current Status

This revised plan reflects the working Go2 ArUco course as confirmed during the student-style setup trial run. The course remains focused on the Unitree Go2. G1 and A2 adaptation ideas are intentionally excluded from this beginner module plan and should be handled later as advanced extension documents.

- Student setup flow was confirmed from a clean GitHub clone.
- requirements.txt is now part of the repo and installs the required packages.
- Marker generation currently creates IDs 0-4 for the beginner course.
- The scripts are run from the scripts/ folder.
- Go2 connection, obstacle avoidance check, camera feed, marker detection, and autonomous patrol all worked in the trial run.

Course Concept

Students learn how to use computer vision to control a Unitree Go2 robot dog with printed ArUco markers. The markers act as visual command signs that the robot can read through its camera.

Go2 camera sees marker
OpenCV identifies marker ID
Python maps that ID to a robot behavior
Go2 performs the behavior

The final project is a visual command system for a mobile robot. The robot can patrol a space, scan for signs, react to marker IDs, and use built-in obstacle avoidance to reduce collision risk.

Confirmed Student Setup Flow

This is the student-facing setup sequence that should be shown before the ArUco theory modules.

```
cd ~/projects
git clone https://github.com/TroyKaravias/go2-aruco-opencv-course.git
cd go2-aruco-opencv-course
```

```
python3 -m venv venv
source venv/bin/activate
```

```
pip install --upgrade pip
pip install -r requirements.txt
```

requirements.txt

The course repository should include a simple requirements.txt file:

```
opencv-contrib-python
numpy
aiortc
unitree-webrtc-connect
```

Confirmed Verification Sequence

Step	Command / Test	Expected Result
1	mkdir -p ~/projects	Projects folder exists.
2	git clone repository	Repository downloads successfully.
3	python3 -m venv venv; source venv/bin/activate	Virtual environment activates.
4	pip install --upgrade pip	pip upgrades inside venv.
5	pip install -r requirements.txt	Required packages install.
6	Python import test	OpenCV, NumPy, and unitree_webrtc_connect import.
7	ArUco support test	cv2.aruco and DICT_4X4_50 work.
8	python scripts/generate_markers.py	Markers 0-4 are generated.
9	ping 192.168.12.1	Go2 network connection responds.
10	python scripts/go2_obstacle_avoidance_check.py	Obstacle avoidance reports enabled=True.
11	python scripts/go2_aruco_scan.py	Camera feed and marker detection work.
12	python scripts/go2_aruco_autonomous.py	Autonomous demo works.

Functional Use Cases

Security or Patrol Robot

- The Go2 patrols a hallway, lab, or room.
- Markers act as signs that tell the robot what to do at different locations.
- Examples: stop near a restricted area, sit at a checkpoint, turn around near a corner marker.

Classroom Robotics Demo

- Students print markers and immediately see the robot respond.
- This makes computer vision physical and easy to understand.
- Examples: marker 1 to stand, marker 2 to sit, marker 0 to stop.

Lab or Warehouse Navigation Signs

- Markers can be placed around an environment as low-cost robot-readable labels.
- Examples: loading zone marker, inspection area marker, turn-around marker.

Human-Robot Interaction

- A person can show a marker instead of using a joystick or app.
- The marker becomes a simple visual language for robot commands.

Full Course Module Plan

00 - Course Overview

Purpose: Explain the big idea, final project, use cases, repo structure, and confirmed student setup flow.

Main topics:

- Visual command signs for the Go2
- What students will build
- GitHub repository overview
- Final demo preview
- Safety expectations

01 - What Is an ArUco Marker?

Purpose: Introduce ArUco markers as printed visual signs that a robot camera can detect.

Main topics:

- Black-and-white square marker
- Unique marker ID
- Black border and inner pattern
- Why markers are useful in robotics

02 - ArUco Dictionaries and Marker IDs

Purpose: Explain how marker dictionaries work and why the generator and detector must match.

Main topics:

- DICT_4X4_50
- 4x4 internal binary grid
- 50 possible IDs
- IDs become robot commands

03 - Creating and Printing ArUco Markers

Purpose: Generate marker PNGs, print them, and test them reliably.

Main topics:

- Beginner script generates IDs 0-4
- Expected files use the aruco_4x4_50_id_X.png naming pattern
- Keep the full black border
- Use good lighting and large prints
- Markers 5-8 can be an advanced extension later

04 - What Is OpenCV?

Purpose: Explain OpenCV as the computer vision library that processes Go2 camera frames.

Main topics:

- Computer vision basics
- Frame processing

- ArUco detector
- Marker ID returned to Python

05 - Go2 Camera and WebRTC Setup

Purpose: Teach how Python connects to the Go2 camera and robot data channel.

Main topics:

- Robot IP 192.168.12.1
- Virtual environment setup
- requirements.txt install
- WebRTC connection
- Camera callback method
- Connection troubleshooting

06 - First ArUco Scanner

Purpose: Build the first scanner that displays the camera feed and prints marker IDs.

Main topics:

- Run python scripts/go2_aruco_scan.py
- Open camera window
- Detect markers
- Draw boxes
- Print marker IDs
- Test one marker at a time

07 - Marker-Controlled Go2 Actions

Purpose: Map marker IDs to built-in Go2 behaviors.

Main topics:

- Marker 0: StopMove
- Marker 1: StandUp
- Marker 2: Sit
- Marker 3: slow forward, then StopMove
- Marker 4: turn/search behavior
- Always stop before sending an action

08 - Marker Confirmation and Cooldowns

Purpose: Make detection more reliable so commands do not repeat too quickly.

Main topics:

- Confirmation count
- Cooldown timer
- Last-seen tracking
- Prevent repeated triggers
- Stop before action

09 - 360-Degree Marker Scanning

Purpose: Make the robot rotate in place to search for markers around the room.

Main topics:

- Rotate slowly
- Scan camera feed
- Stop when marker appears
- Confirm marker
- Run action

10 - Movement While Scanning

Purpose: Add slow movement while the robot searches, bridging toward patrol.

Main topics:

- Slow forward movement
- Slight turn
- Stop immediately on marker
- Safe testing space
- Marker 0 kept ready as stop marker

11 - Autonomous Patrol Behavior

Purpose: Build the patrol state machine used in the working demo.

Main topics:

- FORWARD state
- SCAN state
- TURN state
- Return to patrol after marker action
- Run python scripts/go2_aruco_autonomous.py

12 - Built-In Go2 Obstacle Avoidance

Purpose: Enable and verify the Go2 built-in obstacle avoidance system.

Main topics:

- Run python scripts/go2_obstacle_avoidance_check.py
- Query obstacle avoidance state
- Enable obstacle avoidance
- Confirm enabled=True
- Explain Python vs robot safety roles

13 - Wall and Corner Recovery

Purpose: Add a failsafe for blocked forward movement near walls or corners.

Main topics:

- Visual motion check
- Detect low camera motion while moving forward

- Back up
- Turn around
- Resume patrol

14 - Full Autonomous ArUco Patrol Script

Purpose: Walk through how the final script combines camera, ArUco detection, robot actions, patrol, and recovery.

Main topics:

- Connection sequence
- Camera callback
- Obstacle avoidance check
- Marker action mapping
- Patrol state machine
- Recovery logic

15 - Final Project Demo

Purpose: Students combine all modules into an ArUco patrol robot dog demonstration.

Main topics:

- Room setup with markers 0-4
- Robot patrols
- Robot reacts to signs
- Students customize marker behaviors
- Safety checklist

16 - Future Improvements and Extensions

Purpose: Show students where the project can go after the beginner course.

Main topics:

- Markers 5-8 for extra actions
- Pose estimation
- More robust obstacle logic
- Dashboard UI as a future project
- Multi-marker missions
- Other Unitree robots as separate advanced ports

Recommended Teaching Path

1. Show the final demo first so students know what they are building.
2. Walk through the GitHub clone and virtual environment setup.
3. Install dependencies with requirements.txt and verify imports.
4. Generate and print markers 0-4.
5. Explain ArUco markers and dictionaries.
6. Connect to the Go2 camera through WebRTC.
7. Detect marker IDs with OpenCV.
8. Trigger safe actions: stop, stand, sit.

9. Add marker confirmation and cooldowns.
10. Add 360-degree scanning.
11. Add slow movement while scanning.
12. Add autonomous patrol.
13. Enable and verify built-in obstacle avoidance.
14. Add blocked-wall recovery.
15. Build the final room patrol demo.

Safety and Testing Notes

- Start with stop, stand, and sit before testing movement.
- Keep the robot on the floor with clear open space around it.
- Use slow movement settings for the beginner version.
- Keep marker 0 available as the stop marker during demos.
- Keep one person ready to stop the script with q or CTRL+C.
- Close the Unitree app, browser UI, and old Python scripts before connecting through WebRTC.
- Treat jump, pounce, wave, shake hands, and heart hands as advanced actions that require extra space.

Current Working Status from Development

- Student-style GitHub clone and install flow worked successfully.
- Go2 camera stream works through unitree_webrtc_connect and WebRTC.
- OpenCV ArUco detection works with DICT_4X4_50.
- Marker generator currently creates markers 0-4.
- Markers 0-4 were detected successfully during development.
- Marker IDs can trigger real Go2 sport commands such as StopMove, StandUp, and Sit.
- Built-in Go2 obstacle avoidance was queried and confirmed enabled=True with code=0.
- The autonomous patrol script is scripts/go2_aruco_autonomous.py.
- The working camera callback method is conn.video.switchVideoChannel(True) and conn.video.add_track_callback(...).
- The patrol system can walk, scan, turn, stop on marker detection, trigger actions, and attempt blocked-wall recovery.

Confirmed Troubleshooting Notes

OpenCV Qt Font Warning

The scanner may show a warning similar to:

```
QFontDatabase: Cannot find font directory ...  
Note that Qt no longer ships fonts.
```

If this warning appears but the camera feed still opens, it can usually be ignored.

WebRTC Connection Timeout

If WebRTC fails or hangs, close anything else connected to the Go2 camera, then clear old scripts:

```
pkill -f go2_aruco
pkill -f april
pkill -f obstacle
pkill -f python
```

Then reconnect to the Go2 Wi-Fi, reactivate the virtual environment, and retry. If it still fails, power-cycle the Go2.

Useful Commands Appendix

Clone the course repository

```
cd ~/projects
git clone https://github.com/TroyKaravias/go2-aruco-opencv-course.git
cd go2-aruco-opencv-course
```

Activate the virtual environment

```
source venv/bin/activate
```

Install dependencies

```
pip install --upgrade pip
pip install -r requirements.txt
```

Test imports

```
python - <<'PY'
import cv2
import numpy as np
from unitree_webrtc_connect.webrtc_driver import UnitreeWebRTCConnection, WebRTCConnectionMethod

print('OpenCV version:', cv2.__version__)
print('NumPy version:', np.__version__)
print('unitree_webrtc_connect import: OK')
print('Install test passed.')
PY
```

Test ArUco support

```
python - <<'PY'
import cv2

print('OpenCV version:', cv2.__version__)

if not hasattr(cv2, 'aruco'):
    raise SystemExit('ERROR: cv2.aruco is missing. Install opencv-contrib-python.')

aruco_dict = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50)
params = cv2.aruco.DetectorParameters()
detector = cv2.aruco.ArucoDetector(aruco_dict, params)
```



```
print('cv2.aruco is available.')
print('DICT_4X4_50 detector created successfully.')
print('ArUco install test passed.')
PY
```

Generate markers

```
python scripts/generate_markers.py
ls aruco_markers
```

Ping the Go2

```
ping 192.168.12.1
```

Check built-in obstacle avoidance

```
python scripts/go2_obstacle_avoidance_check.py
```

Run the basic scanner

```
python scripts/go2_aruco_scan.py
```

Run the autonomous demo

```
python scripts/go2_aruco_autonomous.py
```

Scope Note: G1 and A2

This plan is intentionally Go2-focused. G1 and A2 adaptation should be documented separately after robot-specific connection, camera, and command tests are confirmed. This avoids confusing beginner students with untested multi-robot material.